

Name: Sanskriti

USN: 2VX24UE048

Exp_9

Title: Prim's Algorithm

```
In [2]: n = int(input("Enter number of vertices in the graph: "))

print("Enter the adjacency matrix where each row is separated by spaces:")
graph = []
for i in range(n):
    row = input().split()
    graph.append([int(x) for x in row])

def get_min_key_vertex(key, in_mst):
    return min((v for v in range(len(key)) if not in_mst[v]), key = lambda v: key[v], default=-1)

def Prim_MCST(graph):
    vertices = len(graph)
    parent = [-1] * vertices
    key = [float('inf')] * vertices
    in_mst = [False] * vertices
    key[0] = 0
    total_cost = 0

    for _ in range(vertices):
        u = min((v for v in range(vertices) if not in_mst[v]), key = lambda v: key[v], default=-1)
        if u == -1:
            break
        in_mst[u] = True

        for v in range(vertices):
            if graph[u][v] != 0 and not in_mst[v] and graph[u][v] < key[v]:
                key[v] = graph[u][v]
                parent[v] = u

    total_cost = sum(key[1:])

    return total_cost

print("=====")
min_cost = Prim_MCST(graph)
print("Minimum cost of Spanning tree using Prime Algorithm: ", min_cost)
print("=====")
```

Enter the adjacency matrix where each row is separated by spaces:

=====

Minimum cost of Spanning tree using Prime Algorithm: 99

=====